INVOICE & BILLING PLATFORM

Complete End-to-End Functional, Technical, Process Flow & Screen Requirements

React + TypeScript | ASP.NET Core (.NET 8/current LTS) | SQL Server | Redis | Hangfire/Quartz | Docker/IIS

This document defines a reusable, injectable, multi-tenant billing platform that can be consumed through REST APIs, .NET SDKs and reusable React components. The .NET backend remains the source of truth for financial calculations and workflows.

1. Executive Summary

Objective: Build a centralized reusable billing platform for customers, products/services, quotations, invoices, taxes, discounts, payments, credit notes, PDF generation, notifications, reports, webhooks and audit logging.

Integration: Host applications integrate through REST APIs, .NET SDK, embeddable React components or headless client libraries.

Core Principle: The billing backend is the source of truth for calculations, status transitions, numbering, tax and authorization.

2. Architecture Principles

Clean Architecture: Billing.API, Billing.Application, Billing.Domain, Billing.Infrastructure, Billing.Contracts, Billing.Worker and automated test projects.

Modularity: No host application directly accesses billing database tables.

Financial Accuracy: Use decimal for all monetary values; never float/double.

Multi-Tenant: Tenant and Application context mandatory and enforced in middleware, authorization and query filters.

Scalability: Stateless APIs, distributed cache and independent background workers.

3. User Roles

System Administrator: Global configuration, tenants, applications, roles, templates, taxes, numbering and audit.

Billing Administrator: Customers, products, quotations, invoices, payments, credit notes and reports.

Billing User: Create/manage authorized billing records.

Approver: Approve invoices/quotations where workflow is enabled.

Read Only: View/search/download authorized records.

Integration Service: Machine-to-machine access using scoped credentials.

4. Customer Management

Capabilities: Create, edit, search, activate/deactivate, view history, outstanding balance and statements.

Fields: Customer code, type, name, company, email, mobile, GST/VAT/Tax ID, PAN/registration ID, credit limit, payment terms, currency, status and notes.

Addresses: Multiple billing/shipping addresses with default flags.

Rules: Customer code unique per tenant; historical invoices retain customer snapshots.

5. Product and Service Catalog

Fields: Code, name, description, type, category, unit, default price, currency, tax category, HSN/SAC, discount allowed and status.

Behavior: Invoice selection copies product values into immutable invoice snapshot.

Actions: Create, edit, search, activate/deactivate.

6. Invoice Management

Lifecycle: DRAFT → ISSUED → SENT → PARTIALLY_PAID → PAID. Additional statuses: OVERDUE, CANCELLED and VOID.

Invoice Data: Customer, billing address, dates, currency, line items, discounts, taxes, charges, notes and terms.

Rules: Draft editable; issued financial data protected; corrections use controlled credit notes/void workflows.

Search: Invoice number, customer, status, date range, amount range, overdue and application filters.

7. Calculation Engine

Line Calculation: Gross = Quantity × Unit Price; Discount applied; Taxable Amount = Gross − Discount; Taxes calculated; Line Total persisted.

Invoice Calculation: Subtotal +/− invoice discounts + taxes + charges + rounding = Grand Total.

Rounding: Tenant-configurable precision and rounding strategy.

Snapshots: Persist tax, product, customer and company snapshots when invoice is issued.

8. Tax and GST

Supported: GST, CGST, SGST, IGST, VAT and custom taxes.

Capabilities: Inclusive/exclusive, single/multiple taxes, item/invoice level, effective dates and jurisdiction rules.

India GST: Support intra-state CGST/SGST and inter-state IGST; HSN/SAC support.

History: Tax configuration changes never alter issued invoices.

9. Payments

Methods: Cash, bank transfer, UPI, credit/debit card, cheque, gateway and custom.

Capabilities: Full/partial/multiple payments, allocations, reversals and refunds.

Rules: Payment cannot exceed due amount unless authorized; reversal creates audit history instead of deleting.

Status: Pending, completed, failed, reversed and refunded as applicable.

10. Credit Notes

Capabilities: Full/partial credit, line-level credit, tax adjustment and refund linkage.

Workflow: Draft → optional approval → issued.

Rules: Credit cannot exceed eligible invoice amount.

11. Multi-Tenant Model

Tenant: Organization/customer consuming the platform.

Application: A host application integrated with billing.

Context: TenantId and ApplicationId resolved from validated JWT claims or approved headers.

Isolation: Global query filters plus authorization checks; UI hiding alone is insufficient.

12. Security

Authentication: OIDC/OAuth2 JWT preferred; service credentials/API keys for integrations.

Authorization: Permission-based policies such as Billing.Invoice.Create, Issue, Void and Payment.Record.

Controls: HTTPS/TLS, rate limiting, input validation, secure headers, OWASP practices and secret rotation.

Audit: Record actor, action, old/new values, tenant, application, IP and correlation ID.

13. Database Technical Design

Database: SQL Server with EF Core migrations.

Money: decimal(19,4) recommended.

Dates: datetime2 UTC for timestamps; date for business dates.

Concurrency: RowVersion on editable entities.

Indexes: Tenant-first composite indexes, e.g. TenantId + InvoiceNumber and TenantId + Status + DueDate.

Integrity: Foreign keys, constraints, transactions and no destructive cascade deletes on financial history.

14. Core Database Tables

Master: Tenants, Applications, Users/identity integration, Customers, CustomerAddresses, Products, Categories, Taxes.

Billing: Invoices, InvoiceItems, Payments, InvoicePaymentAllocations, CreditNotes, CreditNoteItems.

Configuration: NumberSequences, InvoiceTemplates, FeatureFlags, WebhookConfigurations.

Operations: AuditLogs, OutboxMessages, WebhookDeliveries, IdempotencyKeys.

15. API Standards

Versioning: /api/v1 routes; breaking changes require a new version.

Errors: ProblemDetails style responses with machine-readable error codes and traceId.

Pagination: Server-side pagination/filtering/sorting with whitelisted fields.

Idempotency: Idempotency-Key for financial create/issue operations.

Documentation: Swagger/OpenAPI and Postman collection required.

16. Core APIs

Customers: POST/GET /api/v1/customers; GET/PUT /customers/{id}; statement endpoint.

Products: POST/GET /api/v1/products; GET/PUT /products/{id}.

Invoices: POST/GET /api/v1/invoices; GET/PUT /{id}; POST /issue, /send, /cancel, /void; GET /pdf.

Payments: POST/GET /api/v1/payments; GET invoice payments; POST reverse.

Credit Notes: POST/GET /api/v1/credit-notes; approval/issue/cancel actions.

Operations: Health, audit, settings, templates, numbering and webhook endpoints.

17. Transaction & Idempotency

Issue Invoice: Transaction validates status, allocates sequence, freezes snapshots, updates status, writes audit and outbox event, then commits.

Record Payment: Validate idempotency key, transactionally create payment/allocation, update due/status and publish outbox event.

Outbox: External side effects occur after database commit to prevent dual-write inconsistencies.

Concurrency: RowVersion conflict returns HTTP 409.

18. Background Jobs

Jobs: Overdue evaluation, reminders, scheduled invoices, PDF generation, reports, email retries and webhook retries.

Technology: Hangfire or Quartz behind an abstraction.

Reliability: Idempotent jobs, retry/backoff, dead-letter/failure visibility.

19. React Technical Architecture

Stack: React + TypeScript, React Router, TanStack Query, React Hook Form + Zod/Yup, Material UI/Ant Design.

Packages: billing-react components; billing-client typed API client; billing-contracts shared contracts.

Rules: UI preview calculations are not authoritative; backend recalculates before save/issue.

Integration: BillingProvider accepts API URL, auth adapter, tenant/application context, locale and feature flags.

Customization: Theme tokens, callbacks and no global CSS collisions.

20. PDF, Email & Webhooks

PDF: Server-side generation from approved templates and issued invoice snapshots.

Email: Provider abstraction, templates, retries and delivery logs.

Webhooks: Event ID, schema version, signed HMAC payload, at-least-once delivery and retry strategy.

Storage: Abstract file storage supporting local, Azure Blob or S3-style providers.

21. Observability

Logging: Structured logs with correlation ID, tenant, application and user context.

Tracing: OpenTelemetry recommended.

Metrics: Latency, errors, invoices issued, payments, job failures and webhook success.

Health: /health/live and /health/ready endpoints.

Monitoring: Dashboard and alerts for API, DB, workers and integrations.

22. Testing

Unit: Calculation engine, state transitions, numbering and permissions.

Integration: Database constraints, transactions, API middleware and idempotency.

E2E: Customer → product → invoice → issue → PDF → partial payment → paid.

Security: Tenant isolation, authorization bypass and injection tests.

Performance: Invoice search and concurrent invoice issuance/payment load tests.

23. Deployment

Environments: Development, QA, UAT/Staging and Production.

CI/CD: Build, test, scan, package, deploy, migrations and smoke tests.

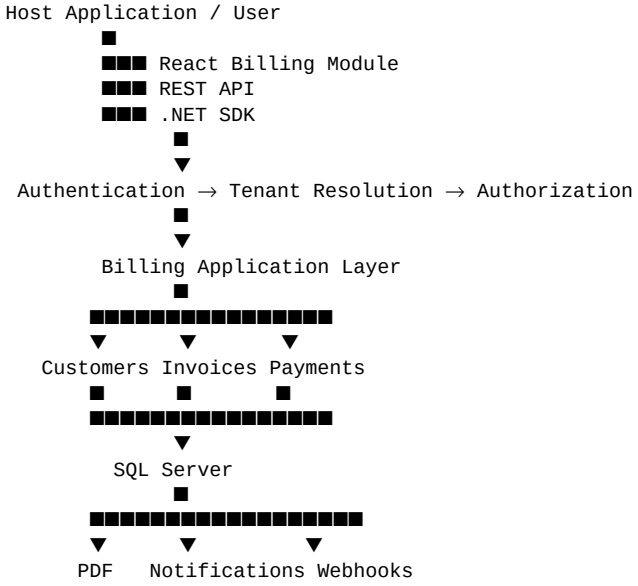
Deployment: IIS initially; Docker/container ready; horizontal API scaling supported.

Secrets: Environment secret store/Key Vault; no secrets in source control.

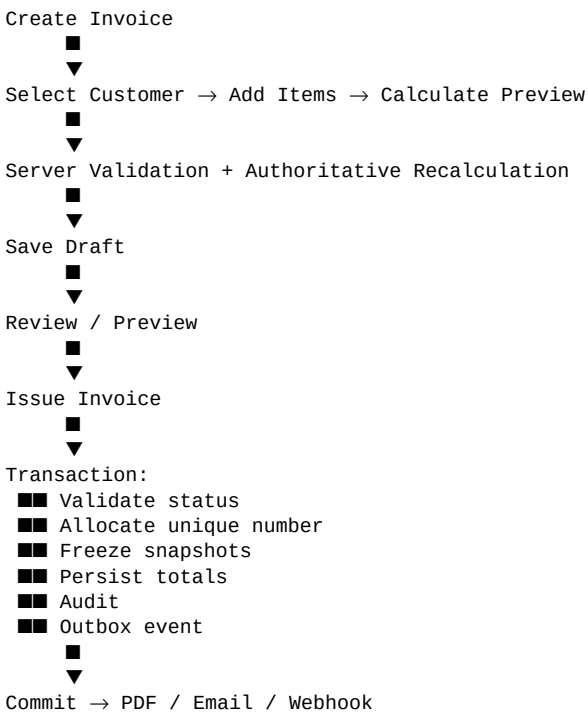
Recovery: Backup/restore runbook, RPO/RTO targets and failed job recovery procedures.

PROCESS FLOW DIAGRAMS

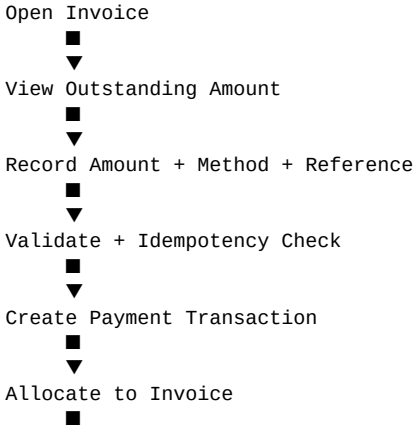
Overall System Flow



Invoice Creation & Issue Flow



Payment Flow



```
     ■■ Due = 0 → PAID
     ■■ Due > 0 → PARTIALLY_PAID
     ■■ Due date passed → OVERDUE
     ■
     ▼
Audit → Notification / Receipt
```

Credit Note Flow

```
Select Issued Invoice
     ■
     ▼
Choose Full / Partial Credit
     ■
     ▼
Select Lines / Amount
     ■
     ▼
Validate Maximum Eligible Credit
     ■
     ▼
Save Draft → Approval (optional)
     ■
     ▼
Issue Credit Note Number
     ■
     ▼
Update Credit / Due Totals
     ■
     ▼
Audit + PDF + Notification
```

Overdue Flow

```
Scheduled Worker
     ■
     ▼
Find Active Invoices
DueDate < Today AND Due > 0
     ■
     ▼
Exclude Paid / Void / Cancelled
     ■
     ▼
Set OVERDUE
     ■
     ▼
Audit / Event
     ■
     ▼
Check Reminder Policy
     ■
     ■■ Not Eligible → End
     ■■ Eligible → Send Reminder → Log → Retry failures
```

DETAILED SCREEN SPECIFICATIONS

1. Billing Dashboard

Purpose: operational billing overview. Widgets: total invoiced, paid, outstanding, overdue, drafts, recent payments and top customers. Filters: date range/application/business unit. Actions: Create Invoice, Record Payment, View Overdue, Reports. APIs: dashboard summary, revenue trend, aging. Acceptance: KPIs reconcile with reports.

2. Invoice List

Columns: invoice number, customer, invoice date, due date, status, currency, total, paid, due and actions. Filters: customer/status/date/overdue/application. Actions: view, edit draft, duplicate, issue, send, PDF, cancel, void and payment. Server-side pagination and debounced search required.

3. Create/Edit Invoice

Sections: customer, metadata, line-item grid, totals, notes/terms, attachments and preview. Fields: dates, currency, PO/reference, configurable project/salesperson. Line grid: product, description, quantity, unit, price, discount, tax, total. Actions: Save Draft, Preview, Issue. Server recalculates all amounts.

4. Invoice Details

Sections: status header, customer snapshot, invoice snapshot, items, tax breakdown, totals, payments, credits, PDF/communication history and audit timeline. Actions based on permission/status. Historical snapshots must be displayed.

5. Customer List & Details

List: code, name, type, tax ID, contacts, outstanding and status. Details tabs: profile, addresses, invoices, payments, statement and audit. Actions: create/edit/activate/deactivate/create invoice.

6. Product/Service Catalog

List: code, name, type, category, unit, price, tax and status. Form: description, HSN/SAC, price, tax category and discount allowed. Inactive products hidden from new invoices.

7. Payment Screens

List: payment number, customer, invoice, amount, method, status, reference and date. Form validates amount and due balance. Supports reversal with reason and audit.

8. Credit Note Screen

Select original invoice, reason, full/partial credit, line selection, amount and tax impact. Workflow draft → approval optional → issued. Credit limit validation mandatory.

9. Tax Configuration

Fields: name, code, rate, type, inclusive/exclusive, priority, effective dates and status. Existing issued invoice tax snapshots never change.

10. Invoice Numbering

Fields: document type, prefix/suffix, tokens, sequence length, reset frequency and preview. Concurrent issuance must remain unique.

11. Template & Branding

Manage logo, company details, colors, header/footer, payment instructions and terms. Support preview, versioning, publish and effective rules.

12. Reports

Invoice register, payments, outstanding, aging, tax summary, customer statement and revenue reports. Filters, charts and CSV/Excel/PDF exports. Large exports asynchronous.

13. Audit Log

Date, user/service, action, entity, ID, tenant/application, correlation ID, IP and result. Detail view shows old/new values with masking.

14. Integration Settings

Application code, allowed origins, service accounts, scopes, webhooks and feature flags. Support secret rotation and integration disablement.

End-to-End Acceptance Journey

Create customer → create product/service → create invoice → calculate tax/discount → save draft → preview → issue unique invoice → freeze snapshots → generate PDF → send invoice → record partial payment → PARTIALLY_PAID → record final payment → PAID → audit and reports reconcile.